Week 5 Discussion

CSE 11 Winter 2026
October 28

Open Discussion Section 5 Interactive Groupwork on Gradescope



Which pumpkin are you today?

Recursion with Objects in java

1. Define an interface
2. Define a base case record
3. Define a recursive record
    - Takes in an object (or objects) of interface type

BOOK EXAMPLE 1:

```
interface Ancestor {}

record Unknown() implements Ancestor {}

record Known(
     String name,
     Ancestor parent1,
     Ancestor parent2) implements Ancestor {}
```

Recursion with Objects in java

1. Define an interface
2. Define a base case record
3. Define a recursive record
    - Takes in an object (or objects) of interface type

BOOK EXAMPLE 2:

```
interface DriveItem {}

record Doc(String name, int sizeKb) implements DriveItem {}

record Folder(
     String name,
     DriveItem[] contents) implements DriveItem{}
```

Recursion with Objects in java - Methods

- Recursive calls are NOT always obvious
- The methods of records should mimic the same pattern as the records themselves!
  - Base case record methods should **NOT** contain recursive calls
  - Recursive case record methods **SHOULD** contain recursive calls

```
interface Ancestor {
 int oldest();
}
record Unknown() implements Ancestor {
 public int oldest() { // how many generations back is the oldest ancestor?
   return 0; // we don't know any of our ancestors
 }
}
record Known(
      String name, Ancestor parent1, Ancestor parent2) implements Ancestor {
 public int oldest() {
   return 1 + Math.max( parent1.oldest(), parent2.oldest() ); // recursive call
 }
}
```

Anatomy of a class

```
class Point {

  private final double x;
  private final double y;

  public Point(double x, double y) {
    this.x = x;
    this.y = y;
  }

  public double x() { return this.x; }
  public double y() { return this.y; }
}
```

- Declare parameters
- Write constructor(s)
  - Why is this constructor public? Because you want other classes to be able to create **Point** objects, including classes in other packages
- Write methods
  - We commonly write getter methods for every parameter

Anatomy of a class - exercise

```
class Point {

  private final double x;
  private final double y;

  public Point(double x, double y) {
    this.x = x;
    this.y = y;
  }

  public double x() { return this.x; }
  public double y() { return this.y; }
}
```

Question: Rewrite this *class* definition as a *record*.

Anatomy of a class - exercise

```
class Point {

  private final double x;
  private final double y;

  public Point(double x, double y) {
    this.x = x;
    this.y = y;
  }

  public double x() { return this.x; }
  public double y() { return this.y; }
}
```

```
record Point(double x, double y) {}
```

Records vs. Classes

	`record`	`class`
Basic purpose	Designed for representing **fixed data**.	More general-purpose Can represent data whose **fields may change.**
Field declaration	Fields are declared directly in the record **header**: `record Point(double x, double y) {}`	Fields are declared **inside** the class body: `class Point { double x; double y; }`
Constructor	Java-provided **canonical constructor** that initializes the fields.	Default constructor takes no arguments. Uses placeholder values. Developer (you) writes other constructors.
Reading fields	**Automatically** gets public accessor methods with the field's name: `p.x()`	Need to write getters/accessors yourself.
Field updates	Cannot update record fields after construction. **IMMUTABLE** 🗿.	Non-`final` fields can be updated
Default `equals()`	Automatically gets an `equals()` that compares the record's fields.	Gets a default `equals()` that effectively checks object identity/reference equality (`==`).

Overloading Methods

- **Overloading:** Giving the exact same name to multiple methods, but distinguishing them by changing the data types or number of inputs they accept.
- ✅ Class or record ***methods*** can be overloaded
- ✅ Class or record ***constructors*** can be overloaded
  - In a record, overloaded constructors **MUST** delegate to another constructor using **`this(...)`**, eventually reaching the canonical constructor.



```
record Point(double x, double y) { }
record RectangleRegion(Point p, double w, double h) {
  RectangleRegion(double w, double h) {
    this(new Point(0, 0), w, h);
  }
}
RectangleRegion r = new RectangleRegion(new Point(1, 2), 4.0, 5.0);
RectangleRegion r2 = new RectangleRegion(4.0, 5.0);
```

- ❌ Overloading does **NOT** work when the only difference is the return types

Overloading Methods

Question: Write a program that instantiates a new **ShoppingCart** object. Then make a call to **addItem** using each of these 4 Overloaded methods.

```
public class ShoppingCart {

    public void addItem(String item) {
        System.out.println("Added 1 " + item);
    }

    public void addItem(double price) {
        System.out.println("Added 1 item for " + price);
    }

    public void addItem(String item, int quantity) {
        System.out.println("Added " + quantity + " " + item);
    }

    public void addItem(String item, int quantity, double price) {
        System.out.println("Added " + quantity + " " + item
            + " for $" + price + " each");
    }
}
```

Overloading Methods

```
public class ShoppingCart {

    public void addItem(String item) {
        System.out.println("Added 1 " + item);
    }

    public void addItem(double price) {
        System.out.println("Added 1 item for " + price);
    }

    public void addItem(String item, int quantity) {
        System.out.println("Added " + quantity + " " + item);
    }

    public void addItem(String item, int quantity, double price) {
        System.out.println("Added " + quantity + " " + item
            + " for $" + price + " each");
    }
}
```

```
ShoppingCart cart = new
ShoppingCart();

cart.addItem("apple");

cart.addItem(2.99);

cart.addItem("apple", 3);

cart.addItem("apple", 3, 0.75);
```

Overloading Methods

```
public class ShoppingCart {

    public void addItem(String item) {
        System.out.println("Added 1 " + item);
    }

    // ...
}
```

❌ **Cannot overload the return type:**

```
public double addItem(String item) { … }
```

Don’t do this!

Overloading Constructors

❌ Cannot do:

```
public Student(int x) { ... }

public Student(int y) { ... }
```

```
public class Student {
    private String name;
    private int id;

    public Student(String name) {
        this.name = name;
    }
    public Student(int id) {
        this.id = id;
    }
    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }
}
```

→ `Student s1 = new Student("Alex");`

→ `Student s2 = new Student(123);`

→ `Student s3 = new Student("Alex", 123);`

Overloading Methods: Exercise

Put your answer to this question in the Gradescope submission!

Question: What method headers would we need for this code to compile? (assume `placeOrder` does not return anything)

```
PizzaOrder order = new PizzaOrder();

order.placeOrder();

order.placeOrder("large");

order.placeOrder(2);

order.placeOrder("large", 2);

order.placeOrder(2, 12.99);
```



Think about

- What parameter types would each method need?
- Can two methods have the same name and number of parameters?

Overloading Methods: Exercise

Question: What method headers would we need for this code to compile? (assume `placeOrder` does not return anything)

```
PizzaOrder order = new PizzaOrder();

order.placeOrder();

order.placeOrder("large");

order.placeOrder(2);

order.placeOrder("large", 2);

order.placeOrder(2, 12.99);
```

```
public void placeOrder()

public void placeOrder(String size)

public void placeOrder(int toppings)

public void placeOrder(String size, int
toppings)

public void placeOrder(int quantity, double
price)
```

Think about

- What parameter types would each method need?
- Can two methods have the same name and number of parameters?

Overloading Methods: Exercise

Based on these method headers:

```
public void placeOrder()

public void placeOrder(String size)

public void placeOrder(int toppings)

public void placeOrder(String size, int
toppings)

public void placeOrder(int quantity, double
price)
```

Which of the following method calls would compile?:

```
order.placeOrder(2.0);

order.placeOrder(2, 12);

order.placeOrder(2.0, 12);
```

Overloading Methods: Exercise

Based on these method headers:

```
public void placeOrder()

public void placeOrder(String size)

public void placeOrder(int toppings)

public void placeOrder(String size, int
toppings)

public void placeOrder(int quantity, double
price)
```

Which of the following method calls would compile?:

~~`order.placeOrder(2.0);`~~

`order.placeOrder(2, 12);`

~~`order.placeOrder(2.0, 12);`~~

Overloading Constructors: Exercise

Which **constructors** are required to support these `new` expressions?

Can they all coexist? Are any unnecessary? **Explain**.

```
new MovieTicket()

new MovieTicket("Dune")

new MovieTicket(12)

new MovieTicket(12.50)

new MovieTicket("Dune", 12.50)
```

A. `public MovieTicket()`

B. `public MovieTicket(String title)`

C. `public MovieTicket(int age)`

D. `public MovieTicket(double price)`

E. `public MovieTicket(String title, double price)`

F. `public MovieTicket(String title, int price)`

G. `public MovieTicket(double age)`

Put your answer to this question in the Gradescope submission!

Overloading Constructors: Exercise

Which **constructors** are required to support these `new` expressions?

Can they all coexist? Are any unnecessary? **Explain**.

```
new MovieTicket()

new MovieTicket("Dune")

new MovieTicket(12)

new MovieTicket(12.50)

new MovieTicket("Dune", 12.50)
```

A. `public MovieTicket()`

B. `public MovieTicket(String title)`

C. `public MovieTicket(int age)`

D. `public MovieTicket(double price)`

E. `public MovieTicket(String title, double price)`

F. `public MovieTicket(String title, int price)`

G. `public MovieTicket(double age)`

D and G cannot coexist because they have the same constructor signature MovieTicket(double)

F is unnecessary

C is optional depending on what you want `new MovieTicket(12)` to do

C+D and E+F CAN coexist because they have different parameter lists

Records vs. Classes (cont'd)

	`record`	`class`
Basic purpose	Designed for representing **fixed data**.	More general-purpose Can represent data whose **fields may change.**
Field declaration	Fields are declared directly in the record **header**: `record Point(double x, double y) {}`	Fields are declared **inside** the class body: `class Point { double x; double y; }`
Constructor	Java-provided **canonical constructor** that initializes the fields.	Default constructor takes no arguments. Uses placeholder values. Developer (you) writes other constructors.
Reading fields	**Automatically** gets public accessor methods with the field's name: `p.x()`	Need to write getters/accessors yourself.
Field updates	Cannot update record fields after construction. **IMMUTABLE** 🗿 .	Non-`final` fields can be updated
Default `equals()`	Automatically gets an `equals()` that compares the record's fields.	Gets a default `equals()` that effectively checks object identity/reference equality (`==`).

1. **Write a class to represent a Dog with fields `boolean isHungry` and `String breed`.**
2. **Should this be implemented as a record or class? Why? Discuss in terms of each of the above points in the record vs. class table.**

Some Important Java field modifiers

- **`public`** – Accessible from anywhere.
- **`private`** – Accessible only within the enclosing class.
- **No modifier (package-private)** – Accessible only within the same package.
- **`static`** – Belongs to the class rather than individual objects.
- **`final`** – Cannot be reassigned after initialization.

TODO

- **PA01 resubmission is due this Friday, Oct 30, if you would like to improve your score**

Thank You!
Any Questions?